PROG1384

Développement Web côté serveur



ccnb.ca

Ajax : envoyer de l'information à PHP

GET, POST et `XMLHttpRequest`

Objectif : comprendre comment une page HTML peut envoyer une valeur à un fichier PHP sans recharger toute la page.

Le scénario

On a une page web avec JavaScript.

On veut envoyer une information à un fichier PHP.

Le fichier PHP reçoit l'information, fait un traitement et retourne une réponse.

Ensuite, JavaScript affiche la réponse dans une partie de la page.

La méthode `open`

Avec `XMLHttpRequest`, la méthode `open` prépare la requête.

```
xhr.open("GET", "fichier.php", true);
```

Les trois informations importantes sont :

1. la méthode HTTP : `GET` ou `POST`;
2. le fichier à appeler;
3. `true` pour dire que l'appel est asynchrone.

Méthode GET

Avec `GET`, les données sont ajoutées directement dans l'URL.

Exemple :

```
fichier.php?deptno=10
```

La fonction `send` reçoit `null`, car l'information est déjà dans l'URL.

```
xhr.send(null);
```

Exemple GET : JavaScript

```
let xhr = new XMLHttpRequest();

xhr.open("GET", "departement.php?deptno=10", true);

xhr.onload = function () {
  $("#resultat").html(xhr.responseText);
};

xhr.send(null);
```

Ici, la valeur `10` est envoyée au fichier `departement.php`.

Exemple GET : PHP

```php
<?php
$deptno = $_GET['deptno'];

...
echo ...
?>
```

Le tableau `$_GET` contient les données envoyées dans l'URL.

La réponse retournée par PHP devient `xhr.responseText`.

Méthode POST

Avec `POST`, les données ne sont pas ajoutées à l'URL.

Les données sont envoyées dans le corps de la requête.

Il faut aussi préciser le format envoyé avec un header.

```
xhr.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
```

Exemple POST : JavaScript

```
let xhr = new XMLHttpRequest();

xhr.open("POST", "departement.php", true);

xhr.setRequestHeader("Content-type", "application/x-www-form-urlencoded");

xhr.onload = function () {
  $("#resultat").html(xhr.responseText);
};

xhr.send("deptno=10");
```

Exemple POST : PHP

```php
<?php
$deptno = $_POST['deptno'];
...
echo ...
?>
```

Le tableau `$_POST` contient les données envoyées avec `send`.

Le PHP peut retourner du texte ou du HTML simple.

Afficher la réponse dans la page

La réponse du serveur se trouve dans :

```
xhr.responseText;
```

Avec JavaScript :

```
document.getElementById("resultat").innerHTML = xhr.responseText;
document.querySelector("#resultat").innerHTML = xhr.responseText;
```

Avec jQuery :

```
$("#resultat").html(xhr.responseText);
```

À retenir

`GET` envoie les données dans l'URL.

`POST` envoie les données avec `send`.

PHP lit les données avec `$_GET` ou `$_POST`.

PHP retourne une réponse avec `echo`.

JavaScript place la réponse dans la page sans rechargement complet.